



DEPARTMENT OF CSE

PROJECT ABSTRACT SUBMISSION

COURSE:– ADAPTIVE SOFTWARE ENGINEERING (24CI3201) A.Y. 2026 - 2027

TITLE OF THE PROJECT	Intelligent CI/CD Pipeline Bottleneck Detection and Optimization System	
S. No.	University ID	Name
1	2420030636	V Praneeth
2	2420030602	G Nisrith Reddy
3	2420030656	Vivek Akhil Reddy K
Name of the Guide		Chandra Shekhar R

Abstract

- The **Intelligent CI/CD Pipeline Bottleneck Detection and Optimization System** aims to improve software delivery efficiency by automatically identifying stages that cause excessive execution time, resource usage, and repeated failures. The system analyzes pipeline data such as build duration, test execution time, queue delays, stage dependencies, and resource utilization to identify recurring bottlenecks. It then provides data-driven recommendations for optimizing pipeline execution and presents the findings through a centralized dashboard integrated with the CI/CD workflow.

Detailed Project Points

1. Pipeline Data Collection

- Collect execution data from CI/CD pipelines.
- Record build, test, deployment, and queue durations.
- Track pipeline success and failure rates.
- Capture CPU, memory, and other available resource metrics.
- Maintain historical execution data for comparison.

2. Bottleneck Detection

- Identify stages with unusually high execution times.
- Detect stages that frequently cause pipeline delays.
- Find repeated failures or unstable pipeline stages.
- Analyze dependencies between pipeline stages.
- Compare current execution with historical performance.

3. Performance Analysis

- Calculate metrics such as:
 - Total pipeline duration
 - Stage execution time
 - Queue/waiting time
 - Resource utilization
 - Failure frequency
 - Throughput
- Identify the stages contributing most to total pipeline time.
- Detect performance degradation over multiple executions.

4. Intelligent Optimization

- Recommend parallel execution for independent stages.
- Suggest optimization of slow test suites.
- Identify inefficient pipeline stages.
- Recommend resource adjustments where appropriate.
- Suggest changes based on historical pipeline behavior.
- Provide a reason for each recommendation rather than only displaying a score.

5. CI/CD Integration

- Integrate with a platform such as GitHub Actions or GitLab CI.
- Automatically collect information after pipeline execution.
- Analyze new pipeline runs without manual data entry.
- Optionally provide a quality/performance warning when a significant regression is detected.

6. Dashboard

- Display overall pipeline performance.
- Show the slowest stages.
- Visualize execution-time trends.
- Display bottleneck severity.
- Show failure and resource-utilization trends.
- Provide optimization recommendations.
- Compare pipeline performance before and after optimization.

7. Validation and Evaluation

- Create sample pipelines containing intentional bottlenecks.
- Measure execution time before optimization.
- Apply the system's recommendations.
- Run the pipeline again.
- Compare:
 - Total execution time
 - Individual stage duration
 - Resource usage
 - Failure rate
- Demonstrate whether the recommended optimization actually improves the pipeline.

8. ASE Concepts Demonstrated

- Software performance engineering
- Software quality measurement
- Continuous integration and continuous delivery
- Software testing
- Software maintenance and evolution
- Architecture and modular design
- Dependency analysis
- Design patterns
- Requirements engineering
- Automated testing
- Configuration management
- Empirical software metrics
- Continuous improvement

Signature of the Guide

HOD